MODULE 3
CHAPTER 1---AGILE DEVELOPMENT

1.1 WHAT IS AGILITY?

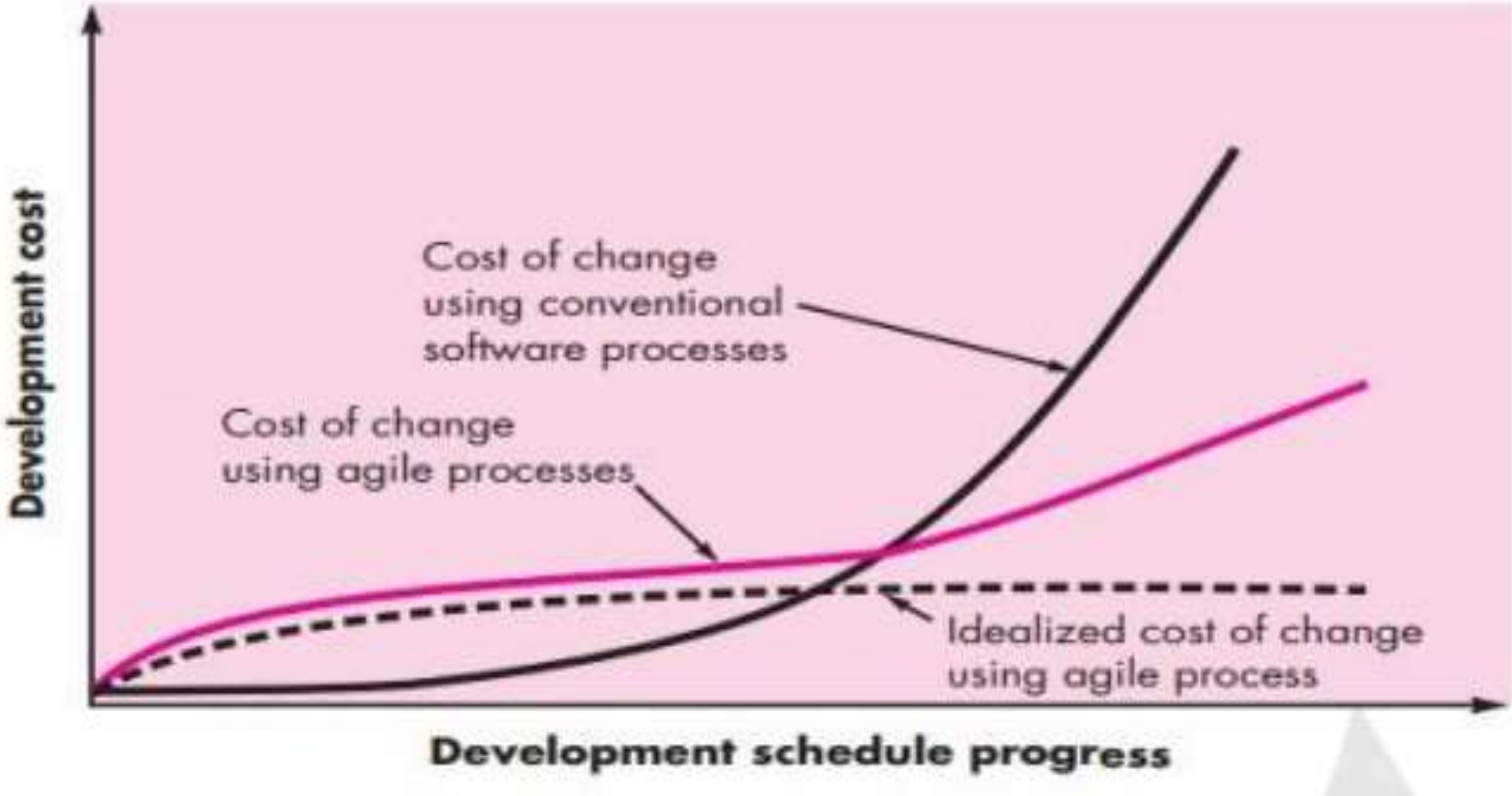
Agile is a software development methodology to build software incrementally using short iterations of 1 to 4 weeks so that the development process is aligned with the changing business needs.

- **Small Steps:** Software is developed in small parts called iterations, usually lasting 1–4 weeks.
- **Accept Changes:** The team can change the software when customer needs or technology changes.
- **Teamwork:** Team members work together and share ideas to develop better software.
- **Quick Delivery:** Working software is delivered to the customer quickly for testing and feedback.
- **Example:** In a food ordering app, the team first develops the login page, then the menu page, and finally the payment page.

AGILITY AND THE COST OF CHANGE

- **Conventional Development:** The cost of making changes increases as the project moves forward.
- **Agile Development:** Agile reduces the cost of changes by developing and delivering software in small parts called increments.
- **Cost of Change Curve:** Agile makes the cost of change curve flatter, so changes can be made even at later stages with less cost and time.
- **Agile Practices:** Continuous testing and pair programming help find errors early and reduce the cost of changes.

Example: If a customer asks to change the payment method, Agile makes it easier and less costly to update the software.


Development cost
Cost of change
using conventional
software processes
Cost of change
using agile processes
Idealized cost of change
using agile process
Development schedule progress

WHAT IS AN AGILE PROCESS?

An **Agile Process** is a software development method that allows the team to make changes easily whenever needed.

Main Characteristics:

1. **Changing Requirements:** Customer requirements may change during software development. It is difficult to predict all requirements in advance.

2. **Design and Construction:** Designing and developing software are done together. The design is checked while developing the software.

3. **Unpredictable Activities:** Analysis, design, development and testing cannot always be planned exactly in advance.

How does Agile handle these changes?

- Agile follows an adaptable process, which means it can adjust to changes.
- Software is developed and delivered in small parts called increments.
- Customers check each part and give feedback.
- The team uses customer feedback to make necessary changes.
- Software is delivered in short periods so that changes can be made quickly.

Example: A team develops a food ordering app. First, they deliver the login page. The customer checks it and gives suggestions. The team makes changes and then develops the next part.

1.3.1 AGILITY PRINCIPLES

Agility principles are basic rules that help a software team develop good software and satisfy customers.

1. **Customer Satisfaction:** The main aim is to satisfy customers by delivering useful software early and regularly.
2. **Accept Changes:** Accept changes in customer requirements, even at the later stages of development.
3. **Frequent Delivery:** Deliver working software regularly in short periods, preferably every few weeks.
4. **Teamwork:** Business people and software developers must work together daily.
5. **Motivated Individuals:** Give team members a good working environment and support. Trust them to complete their work.
6. **Face-to-Face Communication:** Team members should communicate directly to share information clearly.

7.Working Software: Completed working software is the main way to measure project progress.

8.Continuous Development: The team should work at a steady speed without too much pressure.

9.Good Design: Maintain good software design and technical quality to make changes easier.

10.Simplicity: Keep the work simple and avoid unnecessary work.

11.Self-Organizing Teams: Team members should plan and organize their own work to get the best results.

12.Regular Improvement: The team should regularly discuss mistakes and find ways to improve their work.

1.3.2 THE POLITICS OF AGILE DEVELOPMENT

- There is a debate about whether Agile development is better than traditional software development.
- Traditional methods give more importance to documents, while Agile gives more importance to working software.
- There are different Agile models, and each model follows a different way of developing software.

1.3.3 HUMAN FACTORS

Agile development gives importance to people, their skills and teamwork. The process is adjusted according to the needs of the team.

Important Human Factors:

1. **Competence:** Team members should have the required skills, talent and knowledge to develop software. They should also learn the Agile process.

2. **Common Focus:** All team members should work towards one common goal – delivering working software to the customer on time.

3. **Collaboration:** Team members should work together, share information and communicate with customers and other people involved in the project.

4. **Decision-Making Ability:** The team should have the freedom to make its own decisions about technical work and project activities.

5. **Fuzzy Problem-Solving Ability:** Team members should be able to handle unclear problems and changing requirements during software development.

6. **Mutual Trust and Respect:** Team members should trust and respect each other. This helps them work together effectively.

7. **Self-Organization:** The team should organize its own work, decide how to follow the process and prepare a schedule to complete the work on time.

1.4 EXTREME PROGRAMMING (XP)

Extreme Programming (XP) is one of the most widely used Agile software development methods.
It focuses on customer needs and delivering working software in small parts.
It uses continuous testing and improvements.
XP was proposed by **Kent Beck in the late 1980s**.

1.4.1 XP VALUES

XP follows five important values:

1. Communication

- Developers and customers should communicate regularly.
- They should share ideas and information clearly.
- It avoids unnecessary lengthy documents.

2. Simplicity

- Develop only what is needed at present.
- Avoid adding unnecessary features.
- Improve the design later whenever needed.

3. Feedback

- Get feedback from customers, team members and software testing.
- Test each part of the software regularly.
- Unit testing is the main testing method used in XP.

4. Courage

- Developers should be ready to accept changes.
- They should have the confidence to improve the software whenever needed.
- They should focus on present requirements and handle future changes when they occur.

5. Respect

- Team members should respect and trust each other.
- They should support each other and work together.
- Successful software delivery increases respect among team members.

1.4.2 THE XP PROCESS

- Extreme Programming (XP) uses an **Object-Oriented Approach** to develop software.
- XP follows a set of rules and practices during software development.

FIGURE 3.2

The Extreme Programming process

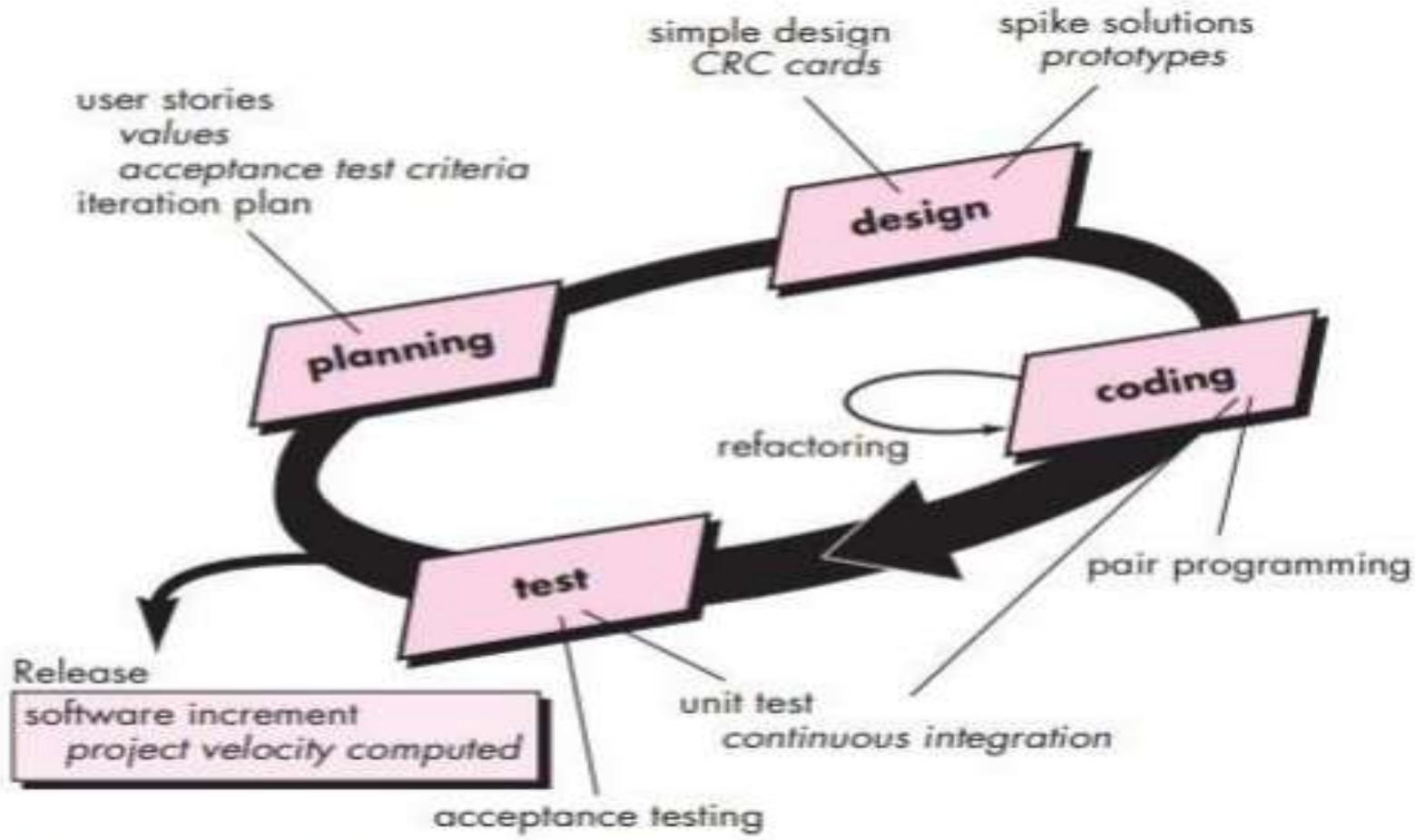


Fig : The Extreme Programming process

KEY XP ACTIVITIES

XP has four main activities: Planning, Design, Coding and Testing.

1. Planning

- Planning starts by listening to the customer and collecting requirements.
- Requirements are written as small descriptions called **User Stories**.
- Each story is written by the customer on an index card.
- The customer gives priority to each story based on its business value (importance).
- The XP team estimates the time needed to complete each story in development weeks.
- If a story takes more than 3 weeks, it is divided into smaller stories. Its priority and time are decided again.
- New stories can be added at any time.
- Stories with the highest priority are developed first.

Example: In a food ordering app, login, food selection and payment are written as user stories. The most important feature is developed first.

2. Design

- XP follows the **Keep It Simple** principle. It means keeping the design simple.
- If a difficult design problem occurs, the team creates a small working model to find a solution. This is called a **Spike Solution**.
- **Refactoring** means improving the internal structure of the code without changing how the software works.

Example: Improving the code of a payment page without changing its working.

3. Coding

- After preparing user stories and the basic design, developers prepare **Unit Tests** for each story in the current software release.
- Developers write code to pass these tests.
- After coding, the code is tested immediately, giving feedback to developers.
- **Pair Programming:** Two developers work together on one computer to write code.
- After completing their work, their code is combined with the work of other team members.

Example: Two developers work together to write the login code.

4. Testing

- Unit tests are prepared before coding starts.
- Automated testing is used to run tests automatically.

Regression Testing: Whenever code is changed, old tests are run again to check whether existing features still work.

- All unit tests are combined into a **Universal Testing Suite**.
- Integration and validation testing are performed daily to check the complete system and find problems early.
- Finding and fixing small problems regularly takes less time than fixing big problems at the last moment.

XP Acceptance Tests (Customer Tests):

- These tests are specified by the customer.
- They check the overall features and functions of the software.
- They check whether the software meets customer requirements.
- They are prepared from user stories completed in a software release.

Example: The customer checks whether login, food selection and payment work correctly in the food ordering app.